



[BIT703 Web Technologies](#) / [Contents](#) / Topic 7: Client-side web application development and programming – JavaScript / Understanding JavaScript

JavaScript AJAX 15m

AJAX stands for **Asynchronous JavaScript and XML**. This is used to make asynchronous communication with the server. AJAX is used to read data from the server and update the page or send data to the server without affecting the current client page.

Facebook uses AJAX!

When you like or comment on a friend's Facebook post, your friend's post will update to show your like or comment, but there is no need for the entire web page to refresh; this is made possible by AJAX.

How to call AJAX in JavaScript

AJAX calls are **one method to load personalised content separately from the rest of the HTML document**. This allows for the entire HTML document to be cached, improving back-end load time.

- Approach 1: Using the XMLHttpRequest object
- Approach 2: Using a framework like jQuery
- Approach 3: Using the fetch() API.

Approach 1: Using the XMLHttpRequest object

In this approach, the XMLHttpRequest object will be used to make an AJAX call.

[Skip to main content](#)

The `XMLHttpRequest()` method will create the `XMLHttpRequest` object. `XMLHttpRequest` objects are used to interact with servers.

The syntax used to create the `XMLHttpRequest` object is:

```
var xhttp = new XMLHttpRequest()
```

The `XMLHttpRequest` object can be used to exchange data with a web server behind the scenes. This means that it is possible to update parts of a web page without reloading the whole page.

All modern browsers support the `XMLHttpRequest` object.

A `XMLHttpRequest` object contains **status** and **statusText** properties, which hold the status of the `XMLHttpRequest` object. The table below explains the `status` and `statusText` properties.

Property	Description
onreadystatechange	Defines a function to be called when the <code>readyState</code> property changes
readyState	Holds the status of the <code>XMLHttpRequest</code> 0: request not initialised 1: server connection established 2: request received 3: processing request 4: request finished and response is ready
status	200: 'OK' 403: 'Forbidden' 404: 'Page not found' For a complete list, go to the Http Messages Reference
statusText	Returns the status-text (for example, 'OK' or 'Not Found')

[Skip to main content](#)

The code below is an example of **approach 1**. It demonstrates how to create a `XMLHttpRequest` object and send a request to the server to exchange data on the client side. Some of the key commands used are `XMLHttpRequest` and `onreadystatechange`.

The code below shows that the `onreadystatechange` function is called every time the `readyState` changes. When `readyState` is 4 and `status` is 200, the response is ready.

Have a look at the code and the output.



[Skip to main content](#)

HTMLCSSJSResultEDIT ON

LIVE

<div id="output">Loading...</div>

Resources1x0.5x0.25xRerun

Example 7.4 Approach 1



Approach 2: Using jQuery

In this approach, jQuery will be used to make an AJAX call. jQuery is a JavaScript framework (or library of functions) that is briefly introduced here but covered later in this topic. The **ajax()** method is used in jQuery to make AJAX calls.

[Skip to main content](#)

The syntax used to create the AJAX call is:

```
$.ajax({arg1: value, arg2: value, ... });
```

or

```
jQuery.ajax({arg1: value, arg2: value, ... });
```

The confusing use of \$ above is actually the reference to the jQuery library, as assigning functions to \$ is valid in JavaScript.

The parameters specify the name:value pairs (that is, the arg1: value pair) that will be used in the AJAX request.

W3Schools has a list of possible name:value pairs that can be used with this approach on the page [jQuery ajax\(\) Method](#).

The code below demonstrates **approach 2**. It has used a configuration file that has defined the parameters URL, type and function to call when a response is received or if an error is received.

Before we have a look at this code, we will briefly discuss the file used. The code uses the uncompressed file:

```
<script src=
"https://code.jquery.com/jquery-3.6.0.min.js">
</script>
```

This uncompressed file is best to use during development or debugging, even though the compressed version of the file saves bandwidth and improves performance in production.

[Skip to main content](#)

When the following code has been successfully executed, it returns a string from the server that we print.

Now, have a look at the code and the output.



HTMLCSSJSResultEDIT ON

LIVE

```
<HTML
script

  src="https://code.jquery.com/jquery
-3.2.1.min.js"></script>

<div id="output">Loading...</div>
```

Resources1x0.5x0.25xRerun

Example 7.5 Approach 2

[Skip to main content](#)



You will learn more about approach 2 using the jQuery ajax() method later, when you learn all about jQuery.

Approach 3: Using the fetch() API

In this approach, the **fetch() API** will be used to make a XMLHttpRequest with the server. Because of its flexible structure, it is easy to use. This API makes a request to the server and gets the result as a Promise, which is resolved to the string.

The syntax is:

```
fetch(url, {config}).then().catch();
```

The fetch() API provides a JavaScript interface for accessing and manipulating requests and responses. It also provides a global fetch() method that provides an easy, logical way to fetch resources asynchronously across the network.

The code below demonstrates **approach 3**. The code uses the fetch() method to make a request to the server to get a URL. Since it is a resolved Promise, we use the then() function and catch() function to create the output for the result. In response, the string will be printed as an output.

Have a look at the code and the output.



[Skip to main content](#)

Example 7.6 Approach 3



We've shown three different ways to make an AJAX request: two using plain JavaScript and one using a library. But which method should you use, and why? Let's start with plain JavaScript. You'd choose plain JavaScript if your code doesn't already use any other JavaScript libraries and you don't want to deal with adding extra files. It's a straightforward option that works well for simple tasks, like a search request, and can be handled directly on the page.

[Skip to main content](#)

Between the two plain JavaScript examples, the first gives you more control over the communication process, while the second is a quicker, simpler method.

As for jQuery (or other libraries), you'd use this approach if jQuery is already being used throughout your web application. In that case, it makes sense to handle AJAX requests with the same library to keep things consistent.



Dig deeper

To learn more about:

- Approach 1: the XMLHttpRequest object method, read *AJAX Full Form* (GeeksforGeeks, 2025).
- Approach 2: the jQuery ajax() method, read *jQuery Ajax() Method* (GeeksforGeeks, 2025).
- Approach 3: the fetch() API method, read *Fetch API in JavaScript* (GeeksforGeeks, 2025).



Thoughts on this page?

